

PRODUCT DOCUMENTATION

Moisture Sorption & Water Activity Estimator

User guide & technical documentation — GAB, BET and Henderson sorption isotherms, two-way conversion between water activity and equilibrium moisture, measured-data fitting scores, a blend equilibrium predictor, and microbial stability bands for shelf-stable foods.

Creator	**Dr. Asif Ahmad**
Product	Moisture Sorption & Water Activity Estimator — version 1.0
Category	Food Science
Published by	Asif SaaS Studio
Date	21 October 2026
Format	Single-file web application (index.html, 35.9 KB)

Private by design: everything runs on the user's device — no uploads, no accounts, no tracking. Honest by design: every result is an estimate from published model equations, and the app says so — commercial release decisions need a measured water activity.

Contents

1. Overview — the stability problem this app solves
2. The science — water activity, sorption isotherms and the monolayer
3. The three models — GAB, BET and Henderson
4. Presets — typical literature parameters
5. Using the calculator
6. Measured data — paste or upload, and how to read RMSE and R^2
7. The blend equilibrium predictor — method and a worked example
8. Microbial stability bands
9. Moisture bases and conversions
10. Honest scope and limitations
11. Outputs — curve CSV and copy summary
12. How correctness was verified — testing summary
13. Adding the product to the Asif SaaS Studio catalogue (Wix CMS)
14. Product information and credits
15. Appendix — the tested core, verbatim

1. Overview — the stability problem this app solves

Whether a shelf-stable food stays safe and acceptable on the shelf is governed less by how much water it contains than by how available that water is — its water activity, a_w. Two products with identical moisture contents can sit at very different water activities, and the one with the higher a_w is the one that grows mould first. The relationship between moisture content and water activity — the sorption isotherm — is therefore one of the most practical curves in food science.

This estimator puts that curve on one screen. It serves food science students meeting isotherms for the first time, small producers formulating dried and intermediate-moisture products, and QA staff who need a defensible first estimate before committing samples to the laboratory. With it you can:

- Convert between water activity and equilibrium moisture content with the three standard models — GAB, BET and Henderson — in both directions and on both moisture bases.
- Start from nine presets of typical literature parameters for common foods, then adjust them.
- Paste or upload your own measured isotherm points and see how well a model fits them (RMSE and R^2).

- Predict the final equilibrium water activity when two to six dry and semi-moist ingredients are blended in a sealed system — and see, in grams, which ingredient gains water and which loses it.
- Read any resulting a_w against the standard microbial stability bands, with the usual non-microbial cautions attached.

> **Estimates, not measurements.** Every number in this app comes from a published model equation with typical parameters. It is a screening and teaching tool. A commercial release decision must rest on a measured water activity (dew-point / chilled-mirror meter) of the actual product. Chapter 10 states the scope in full.

2. The science — water activity, sorption isotherms and the monolayer

2.1 Water activity

Water activity is the ratio of the vapour pressure of water in a food to the vapour pressure of pure water at the same temperature: $a_w = p / p_0$. It runs from 0 (bone dry) to 1 (pure water). Microorganisms respond to a_w, not to moisture percentage, because a_w measures the water they can actually use: most bacteria stop growing below about 0.91, most yeasts below about 0.88, most moulds below about 0.80, and below 0.60 no microbial growth occurs at all (Chapter 8 gives the full band table).

2.2 Sorption isotherms

At a fixed temperature, the equilibrium moisture content of a food plotted against a_w traces a characteristic S-shaped (Type II) curve: a steep rise at low a_w as the first water binds tightly to the solid, a gentler middle section as further layers accumulate, and a sharp upturn at high a_w as loosely held, solvent-like water dominates. An isotherm is measured either by drying a wet sample stepwise (desorption) or by humidifying a dry one (adsorption); the two curves are close but not identical — the difference is hysteresis, which this app does not model (Chapter 10).

2.3 The monolayer

The most important single parameter of an isotherm is the **monolayer moisture content**, M_0: the moisture at which water forms a single tightly bound layer over the food's internal surfaces. Near the monolayer, a food is at its most stable — lipid oxidation is slowest near (slightly above or below) this point, and most deterioration reactions are at their minimum. Both the GAB and BET models return M_0 as a fitted parameter, which is why they remain the workhorses of food stability work.

3. The three models — GAB, BET and Henderson

In all three models, M is the equilibrium moisture content in grams of water per 100 g of dry solids (dry basis), and a_w is water activity.

3.1 GAB (Guggenheim–Anderson–de Boer)

$$M = M_0 \cdot C \cdot K \cdot a_w \,/\, [\, (1 - K{\cdot}a_w) \cdot (1 - K{\cdot}a_w + C{\cdot}K{\cdot}a_w) \,]$$

Parameter	Meaning	Typical range
M_0	Monolayer moisture content (g / 100 g dry solids)	≈ 3–12 for most foods
C	Guggenheim constant — strength of binding in the first layer	≈ 1–25
K	Correction factor for the multilayer (must be < 1)	≈ 0.7–1.0

The GAB model is the most widely used isotherm in food science because it holds across almost the whole practical range, a_w ≈ 0.05–0.95. Two cautions are built into the app: results above a_w 0.95 are flagged, and as $K{\cdot}a_w$ approaches 1 the denominator collapses toward zero and the curve rockets upward — a warning appears at $K{\cdot}a_w \geq 0.95$ and the result is flagged unreliable at ≥ 0.99. The inverse direction (moisture → a_w) is solved analytically through the quadratic form of the equation, with a bisection fallback.

3.2 BET (Brunauer–Emmett–Teller)

$$M = M_0 \cdot C \cdot a_w \,/\, [\, (1 - a_w) \cdot (1 + (C - 1){\cdot}a_w) \,]$$

BET is the classical two-parameter isotherm: M_0 (monolayer) and C (a binding-energy constant). It is theoretically sound only at low water activity, a_w ≈ 0.05–0.45; the app cautions outside that window and treats anything above it as extrapolation. Within its range it is excellent, and it carries a neat identity used in testing (Chapter 12): at $a_w = 1 / (1 + \sqrt{C})$, the equation gives $M = M_0$ exactly. GAB reduces to BET when K = 1.

3.3 Henderson

$$a_w = 1 - \exp(\, -k \cdot M^n \,) \quad \Leftrightarrow \quad M = [\, -\ln(1 - a_w) \,/\, k \,]^{1/n}$$

Henderson's is an empirical two-parameter equation (k, n) long used for grains, pasta and dried products in storage engineering. It has no monolayer concept, but it is algebraically invertible in both directions with no numerical solving, and it fits many cereal products

respectably across the mid-range. A useful check point: when $k \cdot M^n = 1$, $a_w = 1 - e^{-1} \approx 0.6321$.

Model	Parameters	Reliable range	Best use
GAB	M_0, C, K	$a_w \approx 0.05$–0.95	General food isotherms; monolayer estimation
BET	M_0, C	$a_w \approx 0.05$–0.45	Monolayer estimation from low-a_w data
Henderson	k, n	Mid-range, product-specific	Grains, pasta, storage calculations

4. Presets — typical literature parameters

Nine presets ship with the app. They are typical published-style values at about 25 °C, chosen to be representative of each food class — they are **not** measurements of any specific commercial product, and the app labels them in exactly those terms. Loading a preset fills the model selector and parameter fields; every value stays editable.

Preset	Model	Parameters
Wheat flour	GAB	M_0 6.5 · C 12 · K 0.85
Skim milk powder	GAB	M_0 5.5 · C 10 · K 0.92
Milled rice	GAB	M_0 6.0 · C 8 · K 0.80
Raisins	GAB	M_0 8.0 · C 4 · K 0.93
Corn starch	GAB	M_0 9.5 · C 12 · K 0.78
Instant coffee	GAB	M_0 5.0 · C 8 · K 0.90
Cracker / biscuit	GAB	M_0 4.5 · C 6 · K 0.85
Potato starch	BET	M_0 8.0 · C 20
Pasta / dried noodles	Henderson	k 0.017 · n 1.6

Sanity anchors for the presets (from the shipped code): wheat flour at 10 g/100 g dry basis sits at $a_w \approx 0.49$; raisins at 20 % wet basis sit at $a_w \approx 0.76$ — the right neighbourhoods for those foods at room temperature.

5. Using the calculator

1. **Step 1 — pick a model or a preset.** Choose GAB, BET or Henderson, or load one of the nine presets (Chapter 4); adjust any parameter.

2. **Step 2 — solve in either direction.** Enter a water activity to get the equilibrium moisture content, shown on both dry and wet bases; or enter a moisture content to get the corresponding a_w. The chart redraws live as parameters change, marks the calculated point in red, and the stability readout (Chapter 8) updates for the resulting a_w.
3. **Step 3 — heed the flags.** BET results outside a_w 0.05–0.45, GAB results above 0.95 or with $K \cdot a_w$ near 1, and any input outside its valid range produce inline messages. A flagged number is a prompt to reconsider the model or the data, not an answer.
4. **Step 4 — convert bases whenever needed.** The built-in converter moves between % wet basis and g/100 g dry basis (Chapter 9) so laboratory figures and label figures can be compared like for like.

6. Measured data — paste or upload, and how to read RMSE and R²

If you have measured isotherm points of your own, the app scores the current model against them. Paste rows into the data box or upload a .csv / .txt file; each row is two numbers — water activity and moisture content on a dry basis:

```
aw, moisture_db
0.11, 4.2
0.33, 6.8
0.53, 9.5
```

- **Tolerant parsing.** Comma, semicolon and tab separators are all accepted; a header row is detected and skipped; blank lines are ignored; spaces around numbers are fine.
- **Nothing is silently dropped.** A line that cannot be read — wrong number of columns, non-numeric text, a_w outside (0, 1), negative moisture — is reported back with its line number, so a typo shows up as an error to fix rather than a quietly missing point.
- **Overlay.** Valid points plot on the isotherm chart in gold, over the model curve.
- **RMSE** (root-mean-square error, in g/100 g dry solids) is the typical size of the gap between model and measurement: smaller is better, and it is in the units of your moisture data, so "RMSE 0.4" means the curve is usually within about half a gram per 100 g. **R²** is the fraction of the variation in your points the curve explains: 1.00 is a perfect pass through every point; below ~0.95 for isotherm work, try another model or re-fit the parameters. With very few points both statistics are fragile — treat three points as an illustration, not a fit.
- **Clear data** removes the points and the scores.

7. The blend equilibrium predictor — method and a worked example

7.1 The physical idea

Seal a dry ingredient and a moist ingredient in one pack and water moves: from the ingredient at higher a_w to the one at lower a_w, until both sit at one common equilibrium water activity. The dry component gets wetter; the moist one dries. Predicting that final a_w is the classic intermediate-moisture-food problem — it decides whether the crisp component stays crisp and whether the blend as a whole stays below the mould line.

7.2 The method used

1. Each ingredient contributes a known dry-solids mass (from its mass and wet-basis moisture) and a known amount of water. Total water is fixed — the system is sealed.
2. For any candidate common a_w, each ingredient's own isotherm gives the moisture it would hold at that a_w, hence the water it would hold. Summed over all ingredients, that is the water the blend would contain at that a_w.
3. The equilibrium a_w is the one where "would contain" equals the actual total water. Because total held water rises monotonically with a_w, bisection on $a_w \in (0.001, 0.999)$ finds it to machine precision; the app reports the water-conservation residual as a check figure (in practice $\sim 10^{-12}$ g — zero for any practical purpose).
4. The equilibrium a_w always lies between the driest and wettest ingredients' starting a_w values — a physical necessity the test suite asserts (Chapter 12).

7.3 Worked example — the app's default blend (verified figures)

The blend tool opens with 100 g of wheat flour at 12 % wet basis (GAB preset: M_0 6.5, C 12, K 0.85) and 100 g of raisins at 20 % wet basis (GAB preset: M_0 8, C 4, K 0.93). The figures below were reproduced by running the shipped code, not estimated by eye:

Quantity	Wheat flour	Raisins
Mass / initial moisture (wet basis)	100 g / 12 %	100 g / 20 %
Dry solids / initial water	88 g / 12 g	80 g / 20 g
Initial a_w (from its isotherm)	0.651	0.763
Final moisture (dry basis)	16.23 g/100 g	22.15 g/100 g
Final moisture (wet basis)	13.96 %	18.13 %
Water after equilibration	14.28 g	17.72 g
Change	**gains 2.28 g**	**loses 2.28 g**

Predicted equilibrium $a_w \approx 0.73$. Total water is 32 g before and after (conservation residual $\approx 2 \times 10^{-12}$ g). Read against Chapter 8, a_w 0.73 sits in the xerophilic-mould band: the blend is not microbiologically carefree — left long enough, moulds adapted to low moisture can grow. That is exactly the kind of formulation warning this tool exists to give early: raise the raisin share and the equilibrium a_w climbs further; dry the raisins harder, or accept a shorter shelf life.

8. Microbial stability bands

The app classifies any computed a_w — calculator or blend — into the standard bands below. Boundaries are the widely used textbook values; individual species vary, and the bands describe growth limits, not kill steps: organisms that cannot grow at a given a_w may still survive there.

a_w band	Interpretation
< 0.60	**No microbial growth.** The most stable zone; only chemical and physical change remain.
0.60 – 0.65	Osmophilic yeasts become possible.
0.65 – 0.75	Xerophilic moulds (adapted to low moisture) can grow.
0.75 – 0.80	Most moulds can grow.
0.80 – 0.87	Most yeasts can grow.
0.87 – 0.91	Bacteria, including many pathogens, can grow. Note: Staphylococcus aureus is unusually tolerant and can grow down to about a_w 0.86, so the top of the yeast band is not bacteria-free.
> 0.91	Most bacteria can grow; the product is perishable in microbial terms.

Non-microbial risks do not vanish at low a_w. Lipid oxidation is slowest near the monolayer and accelerates again at very low and at high moisture; enzymatic activity becomes significant above roughly a_w 0.75; and texture failures — loss of crispness, caking, sugar crystallisation — are driven by moisture migration between components (Chapter 7) even when every band says "safe". A stable product needs all three questions answered, not just the microbial one.

9. Moisture bases and conversions

Moisture content is quoted two ways in food work, and confusing them is a classic source of error:

Basis	Definition	Conversion
Wet basis (% wb)	Water as a percentage of the total (wet) mass — the label and specification convention	—
Dry basis (g/100 g db)	Water per 100 g of dry solids — the isotherm convention, used by all three models	$M_{db} = M_{wb} / (100 - M_{wb}) \times 100$

The inverse is $M_{wb} = M_{db} / (100 + M_{db}) \times 100$. Anchor example used in testing: 20 % wet basis is exactly 25 g/100 g dry basis. The app converts everywhere it matters: model inputs and outputs are dry basis, blend ingredient moistures are entered wet basis (as they appear on specifications) and reported on both, and the calculator shows both bases side by side.

10. Honest scope and limitations

- **Estimates only.** Results come from published model equations with typical parameters. They are suitable for teaching, formulation screening and deciding what to measure first.
- **Release decisions need a measurement.** This app is not a substitute for a measured a_w (dew-point / chilled-mirror meter) on the actual product, and it never claims to be.
- **Presets are typical, not specific.** Published-style values for a food class can differ appreciably from your product's fitted parameters — variety, process and composition all move an isotherm.
- **One temperature.** Parameters are temperature-specific; the presets assume about 25 °C and no temperature correction is applied.
- **No hysteresis.** Adsorption and desorption curves differ; the app uses one curve per ingredient, which is the standard first approximation.
- **Blend idealisations.** The predictor assumes a sealed system, full equilibration time, no temperature change, and water as the only thing that moves — no solute migration, no fat barriers, no component interactions. Real packs can equilibrate slowly or unevenly.
- **Model edges.** BET above a_w ~0.45 is extrapolation; GAB results as $K \cdot a_w \to 1$ are unreliable. The app flags both rather than hiding them.
- **No backend, nothing faked.** Every feature runs in the page: no server, no API keys, no accounts, no storage.

11. Outputs — curve CSV and copy summary

- **Download CSV** exports the current isotherm curve: a_w from 0.05 to 0.95 in 0.05 steps, with moisture on dry and wet bases — the table behind the chart, ready for a spreadsheet or a report.
- **Copy summary** appends a plain-text record of a calculator or blend result (model, parameters, inputs, result, stability band) to a copy-ready text area, with a clipboard button and manual copy as fallback — the quickest way to move a result into lab notes or an email.

12. How correctness was verified — testing summary

The app's calculation core was extracted from the shipped file and run in Node against independently hand-computed expectations — the same formulas were never simply called twice. **99 automated checks — all passed.**

- **Hand-computed anchors.** GAB at $M_0 = 5$, $C = 10$, $K = 0.8$, $a_w = 0.5$ gives $20 / 2.76 =$ **7.2464** by literal arithmetic — matched. BET with $M_0 = 5$, $C = 10$ at $a_w = 0.30$ gives **5.79** — matched.
- **The BET monolayer identity.** At $a_w = 1 / (1 + \sqrt{C})$ the BET equation reduces to $M = M_0$ exactly; verified for $C = 16$ ($a_w = 0.2$, $M_0 = 8 \rightarrow 8.000000$) and $C = 9$.
- **Henderson check point.** At $k \cdot M^n = 1$, $a_w = 1 - e^{-1} \approx 0.6321$, in both directions.
- **Round-trips.** $a_w \rightarrow M \rightarrow a_w$ across the full grids: GAB 0.05–0.95 (19 points, tolerance 1e-4), BET 0.05–0.45, Henderson both directions 0.1–0.9.
- **Band classifier boundaries** — 0.599 / 0.60 / 0.65 / 0.75 / 0.80 / 0.86 / 0.87 / 0.909 / 0.91 / 0.99 each land in the stated band.
- **CSV parser** — headers, semicolons, tabs, blank lines, bad lines reported with their line numbers, out-of-range values, single-column rows, empty input, negatives, stray spaces.
- **Fit statistics** — points generated exactly on a curve give $R^2 = 1$ and RMSE = 0; a hand-worked RMSE/R^2 case matches.
- **Blend solver** — identical isotherms return the pooled a_w; gain/loss directions follow the initial-a_w ordering; water-conservation residual < 1e-6 in every case, including a three-ingredient mixed-model blend (GAB + BET + Henderson); the flour + raisin case of Chapter 7 (total water 32 g) is itself a test fixture, and its equilibrium a_w lies strictly between the ingredients' initial values, as physics requires.
- **Syntax.** The full inline script was extracted and passed `node --check`.

13. Adding the product to the Asif SaaS Studio catalogue (Wix CMS)

1. **Step 1.** Wix dashboard → CMS → **SaasProducts** → Add Item.
2. **Step 2.** Title: **Moisture Sorption & Water Activity Estimator**. Category: **Food Science**. Slug: `water-activity-estimator`.
3. **Step 3.** Summary: "Predict equilibrium moisture and blend water activity with GAB, BET and Henderson isotherms — plus microbial stability bands." Version: 1.0. Demo: No.
4. **Step 4.** Paste the full contents of `index.html` into the **appHtml** field.
5. **Step 5.** Set **published** on, save, then open the live app page and run the smoke test: set model = BET with M_0 = 5, C = 10 and calculate moisture at a_w 0.30 → expect ≈ 5.79 g/100 g dry solids; load the wheat flour preset and solve a_w from moisture 10 → expect ≈ 0.49; paste a few measured points and plot them; then run the default flour (12 % wb) + raisins (20 % wb) blend → expect equilibrium a_w ≈ 0.73, with the flour gaining ≈ 2.3 g of water and the raisins losing the same amount.

14. Product information and credits

Field	Detail
Product	Moisture Sorption & Water Activity Estimator
Version	1.0 (21 October 2026)
Creator	**Dr. Asif Ahmad**
Publisher	Asif SaaS Studio
Category / slug	Food Science / water-activity-estimator
Deliverable	index.html — 35,940 bytes (35.9 KB), single file, all CSS and JavaScript inline
Requirements	Any modern browser; no server, no installation, no account
Verification	99 automated checks against the shipped core — all passed (Chapter 12)

Appendix — the tested core, verbatim

The code below is the complete calculation core of the product exactly as it ships inside `index.html`: the three isotherm models and their inverses, the basis converters, the band classifier, the CSV parser, the fit statistics, and the water-conservation blend solver. Interface wiring is omitted.

```
function gabM(p, aw){ var a = p.K * aw; return (p.M0 * p.C * a) / ((1 - a) * (1 - a + p.C * a)); }
function betM(p, aw){ return (p.M0 * p.C * aw) / ((1 - aw) * (1 + (p.C - 1) * aw)); }
```

```
function hendersonM(p, aw){ return Math.pow(-Math.log(1 - aw) / p.k, 1 / p.n); }
function hendersonAw(p, M){ return 1 - Math.exp(-p.k * Math.pow(M, p.n)); }

function modelM(model, p, aw){
  if (model === 'gab') return gabM(p, aw);
  if (model === 'bet') return betM(p, aw);
  return hendersonM(p, aw);
}
/* Solve M(C-1)x^2 + (M0*C - M(C-2))x - M = 0 for x (=K*aw for GAB, =aw for BET); returns x or null */
function quadSolve(p, M){
  var A = M * (p.C - 1), B = (p.M0 * p.C - M * (p.C - 2)), Cc = -M;
  if (Math.abs(A) < 1e-14) { if (Math.abs(B) < 1e-14) return null; return -Cc / B; }
  var disc = B * B - 4 * A * Cc;
  if (disc < 0) return null;
  var sq = Math.sqrt(disc);
  var r1 = (-B + sq) / (2 * A), r2 = (-B - sq) / (2 * A);
  return { r1: r1, r2: r2 };
}
function bisectAw(model, p, M){
  var lo = 1e-9, hi = 0.999999, flo = modelM(model, p, lo) - M, i, mid, fm;
  if (flo > 0) return lo;
  if (modelM(model, p, hi) - M < 0) return hi;
  for (i = 0; i < 200; i++){
    mid = (lo + hi) / 2; fm = modelM(model, p, mid) - M;
    if (Math.abs(fm) < 1e-9) return mid;
    if (fm < 0) lo = mid; else hi = mid;
  }
  return (lo + hi) / 2;
}
function gabAw(p, M){
  if (M <= 0) return 0;
  var q = quadSolve(p, M), cand = [], i, x, aw, err, best = null, bestErr = 1e300;
  if (q !== null){ if (typeof q === 'number') cand.push(q); else { cand.push(q.r1); cand.push(q.r2); } }
  for (i = 0; i < cand.length; i++){
    x = cand[i]; if (!(x > 0)) continue;
    aw = x / p.K; if (!(aw > 0 && aw < 1)) continue;
    err = Math.abs(gabM(p, aw) - M);
    if (err < bestErr){ bestErr = err; best = aw; }
  }
  if (best !== null && bestErr <= 1e-6 * Math.max(1, M)) return best;
  return bisectAw('gab', p, M);
}
function betAw(p, M){
  if (M <= 0) return 0;
  var q = quadSolve(p, M), cand = [], i, x, err, best = null, bestErr = 1e300;
  if (q !== null){ if (typeof q === 'number') cand.push(q); else { cand.push(q.r1); cand.push(q.r2); } }
  for (i = 0; i < cand.length; i++){
    x = cand[i]; if (!(x > 0 && x < 1)) continue;
    err = Math.abs(betM(p, x) - M);
    if (err < bestErr){ bestErr = err; best = x; }
  }
  if (best !== null && bestErr <= 1e-6 * Math.max(1, M)) return best;
  return bisectAw('bet', p, M);
}
function modelAw(model, p, M){
  if (M <= 0) return 0;
  if (model === 'gab') return gabAw(p, M);
  if (model === 'bet') return betAw(p, M);
  return hendersonAw(p, M);
}
function wbToDb(wb){ return wb / (100 - wb) * 100; }
function dbToWb(db){ return db / (100 + db) * 100; }

var BANDS = [
  { max: 0.60, range: 'Below 0.60', verdict: 'Microbiologically stable', color: '#2e7d32',
    desc: 'No microbial growth expected. The most stable zone — shelf-stable products are usually targeted below
aw 0.60.' },
  { max: 0.65, range: '0.60 – 0.65', verdict: 'Osmophilic yeasts possible', color: '#7a9a01',
    desc: 'Osmophilic yeasts may grow slowly; most other microbes cannot.' },
  { max: 0.75, range: '0.65 – 0.75', verdict: 'Xerophilic mould risk', color: '#c9a227',
    desc: 'Xerophilic (dry-tolerant) moulds can grow; shelf life becomes mould-limited.' },
  { max: 0.80, range: '0.75 – 0.80', verdict: 'Mould risk', color: '#d9820f',
    desc: 'Most moulds can grow. Enzymatic activity also becomes significant above about aw 0.75.' },
  { max: 0.87, range: '0.80 – 0.87', verdict: 'Yeast risk', color: '#e07b24',
    desc: 'Most yeasts can grow; many moulds too. Short shelf life without other hurdles.' },
  { max: 0.91, range: '0.87 – 0.91', verdict: 'Bacterial risk — pathogens possible', color: '#c1440e',
    desc: 'Many bacteria, including pathogens, can grow. Note Staphylococcus aureus can grow down to about aw
```

```
0.86.' },
  { max: Infinity, range: 'Above 0.91', verdict: 'High risk — most bacteria', color: '#b3261e',
    desc: 'Most bacteria can grow. Perishable-food territory; refrigeration or other hurdles required.' }
];
function classifyAw(aw){
  for (var i = 0; i < BANDS.length; i++) if (aw < BANDS[i].max) return i;
  return BANDS.length - 1;
}

function parseData(text){
  var points = [], errors = [], lines = String(text == null ? '' : text).split(/\r?\n/), i, raw, parts, a, m,
first = true;
  for (i = 0; i < lines.length; i++){
    raw = lines[i];
    if (raw.trim() === '') continue;
    parts = raw.split(/[;,\t]/);
    if (parts.length < 2){ errors.push({ line: i + 1, text: raw }); first = false; continue; }
    a = parseFloat(String(parts[0]).trim()); m = parseFloat(String(parts[1]).trim());
    if (isNaN(a) || isNaN(m)){
      if (first){ first = false; continue; } /* header row */
      errors.push({ line: i + 1, text: raw }); continue;
    }
    first = false;
    if (!(a > 0 && a < 1) || !(m >= 0)){ errors.push({ line: i + 1, text: raw }); continue; }
    points.push({ aw: a, m: m });
  }
  return { points: points, errors: errors };
}

function fitStats(points, fn){
  var n = points.length, i, pred, ssRes = 0, mean = 0, ssTot = 0;
  if (n === 0) return { n: 0, rmse: NaN, r2: NaN };
  for (i = 0; i < n; i++) mean += points[i].m;
  mean /= n;
  for (i = 0; i < n; i++){
    pred = fn(points[i].aw);
    ssRes += (points[i].m - pred) * (points[i].m - pred);
    ssTot += (points[i].m - mean) * (points[i].m - mean);
  }
  return { n: n, rmse: Math.sqrt(ssRes / n), r2: ssTot > 0 ? 1 - ssRes / ssTot : (ssRes === 0 ? 1 : NaN) };
}

/* ingredients: [{mass (g, wet), wb (% wet basis), model, params}] */
function solveBlend(ings){
  var D = 0, W = 0, det = [], i, d, w;
  for (i = 0; i < ings.length; i++){
    d = ings[i].mass * (1 - ings[i].wb / 100);
    w = ings[i].mass * ings[i].wb / 100;
    D += d; W += w;
    det.push({ dry: d, water0: w, m0db: wbToDb(ings[i].wb) });
  }
  function totalWater(aw){
    var s = 0;
    for (var j = 0; j < ings.length; j++) s += det[j].dry * modelM(ings[j].model, ings[j].params, aw) / 100;
    return s;
  }
  var lo = 0.001, hi = 0.999, mid = lo;
  for (i = 0; i < 300; i++){
    mid = (lo + hi) / 2;
    if (Math.abs(totalWater(mid) - W) < 1e-10) break;
    if (totalWater(mid) < W) lo = mid; else hi = mid;
  }
  var awEq = (lo + hi) / 2, items = [], W1 = 0;
  for (i = 0; i < ings.length; i++){
    var fdb = modelM(ings[i].model, ings[i].params, awEq);
    var w1 = det[i].dry * fdb / 100;
    W1 += w1;
    items.push({
      finalDb: fdb, finalWb: dbToWb(fdb),
      water0: det[i].water0, water1: w1, delta: w1 - det[i].water0,
      initialAw: modelAw(ings[i].model, ings[i].params, det[i].m0db)
    });
  }
  return { aw: awEq, items: items, totalDry: D, totalWater0: W, totalWater1: W1, residual: Math.abs(W1 - W) };
}
```